

-



- [Core Concepts](#)
- Rules

On this page

Rules

Was this page helpful? 

A rule defines a condition based on the fields of a [data source](#) that produces a binary result for each record that matches the condition. The condition to evaluate is defined by coding in [RDL \(Rule Definition Language\)](#).

Defining rules

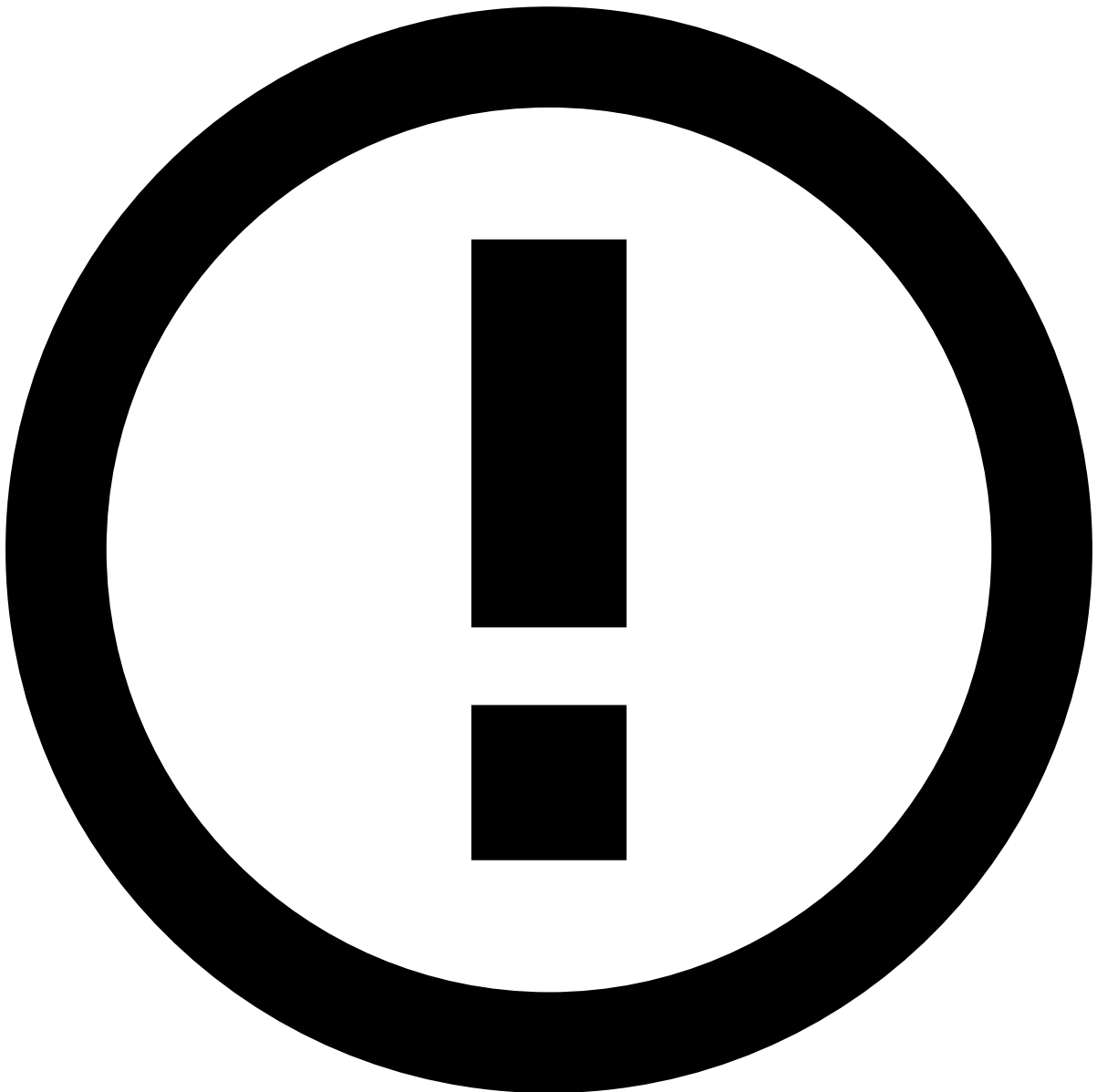
The following image shows how a rule is defined. This example depicts a rule for an account opening scenario to automatically accept the customers' application if their income is significantly above the average for the area where they live.

To fully understand rules in Pulse you must check all details regarding [how a rule is composed](#) and [how it works](#).

Rules projects

Similarly to [models](#), rules are organized into rules projects. Rules projects define which data source field should be available for rules to target, for example, to set all transactions from a specific card automatically as fraud. In addition, rules projects act as logical containers that group rules related with the same context.

A rules project is associated with a data source. Since the rule concept is not directly related to machine learning, it does not involve a training stage. Thus, it is possible to deploy rules directly into Production without having them run over test data. So in fact, a rules project is bound to the schema of a data source and does not depend on the data itself.



info

You can evaluate your rules over any data source as long as it is [compatible](#) with the data schema specified for the rules project.

Target variable

When creating a rules project, you must define the data source on which the rules will operate. In addition, you must choose a **target variable**, which is the field whose value you are trying to detect with your rules. For example, *fraud/not fraud*.

Pulse lets you target any categorical field in the data source schema. In practice, this means that rules can be useful both for binary and multi-class classification.

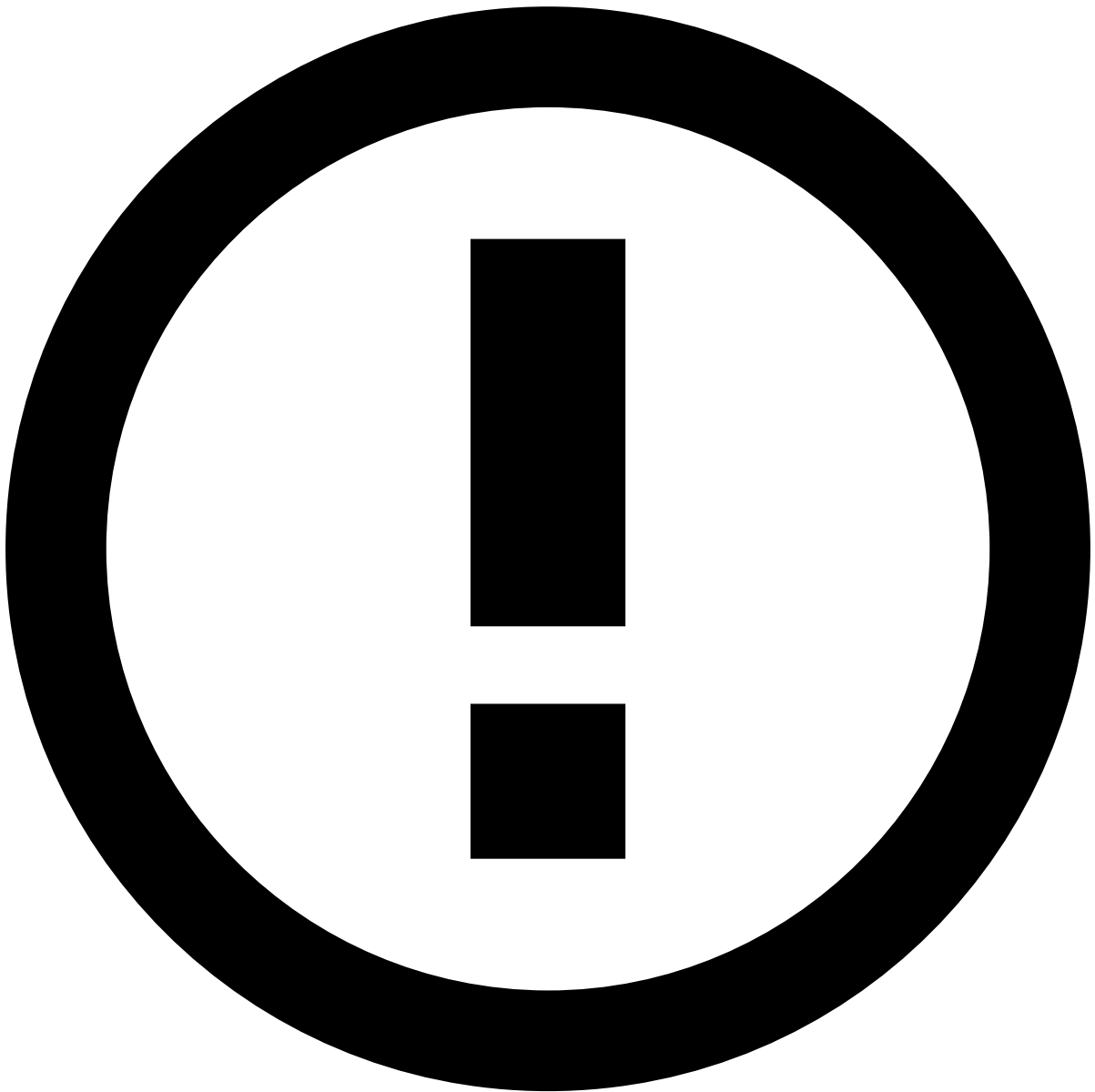
For each rule in the rules project, Pulse allows you to select which of the possible values should be set to the target variable if the rule is triggered. Thus, in the same rules project, you can configure some rules to set transactions as *fraud*, and other rules that set transactions as *not fraud*.

When you evaluate rules in a rule snapshot, Pulse calculates the rule performance for each possible value of the target variable, letting you adjust the resulting visualization accordingly.

State

Rules are usually stateless, but when you cannot use [plans](#) to calculate certain types of windows and add metrics as fields of a schema, Pulse lets you define these windows directly for a rules project.

Therefore, when you need to develop very robust rules that perform aggregations over time windows, you must configure how Pulse should manage their internal state.



info

It is recommended that whenever possible, you use [plans](#) to handle your time-aware operations in data science. The plan will generate data fields with these metrics, making them accessible to rules.

Activating state in your rules project is an advanced operation that must be carefully done and requires that the user understands the full implications of his actions:

- By enabling state in a rules project, Pulse manages an isolated environment where all rules contained in the project will have access to [Pulse Query Language metrics](#) (i.e., aggregations) and [Rule Definition Language variables](#).
- To activate state, the schema in your rules project must include a timestamp field. Otherwise, you cannot perform time-based aggregations.
- When configuring partitioning fields, you must be extremely careful. They define how to distribute processing when generating a rule evaluation. Make sure data is correctly partitioned, otherwise the evaluation results may not make sense.

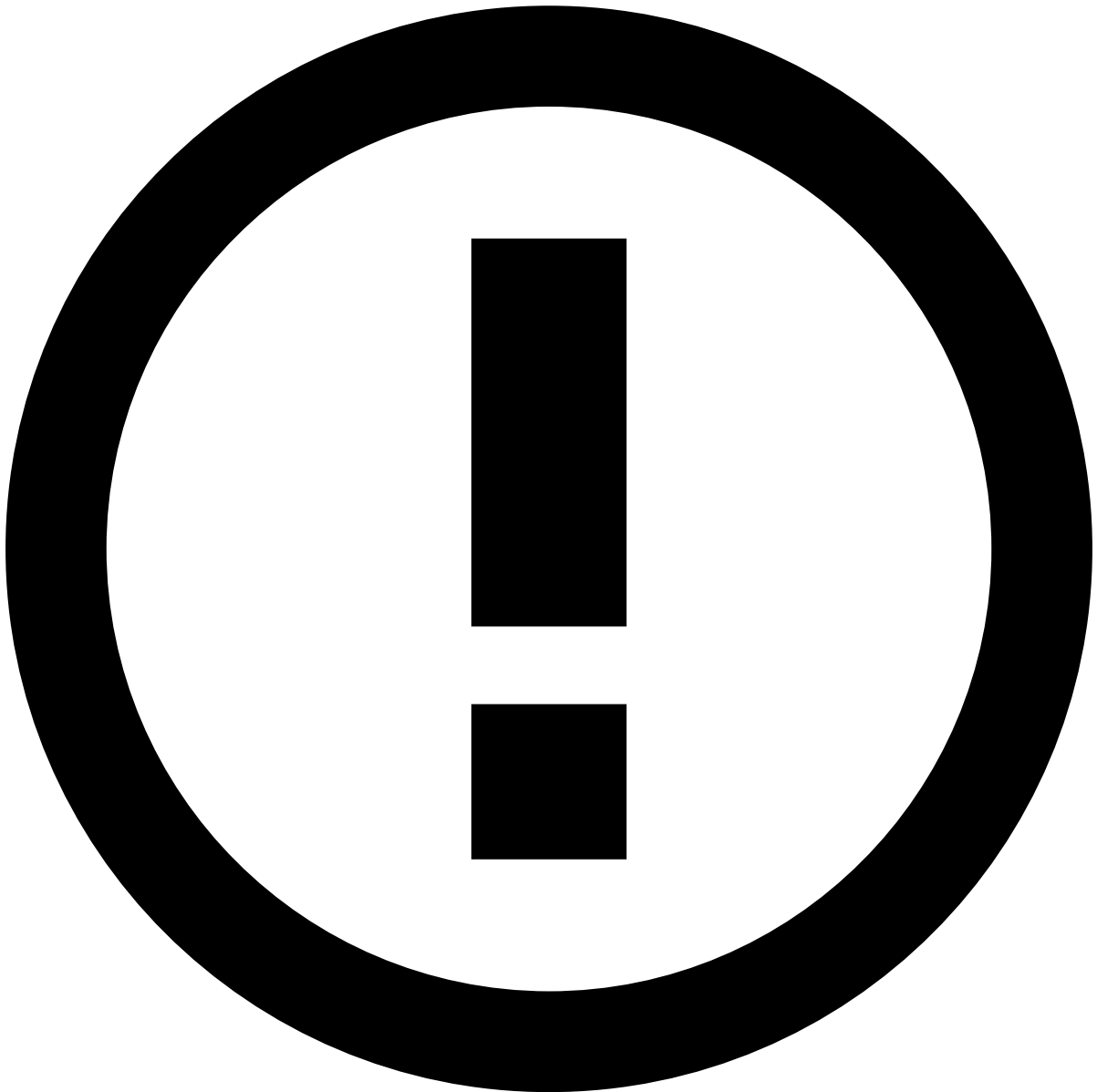
Learn more about state in rules by checking [how a rule is composed](#).

Multi-tenancy📖

Multi-tenancy is the ability to support multiple tenants in the same environment offering a practical solution to apply rules to several tenants at the same time, as well as to customize rules for each tenant.

When creating a rules project, the user can select from the following options:

- **Global** is used for rules that are not multi-tenanted.
- With **Duplicated for Tenant**, the same rule applies to all tenants, but rule conditions can be computed using tenant-specific data.
- With **By Tenant**, custom rules apply to individual tenants.



info

A **landlord** is a client who has a fraud prevention installation that he can offer to other clients, also known as tenants, acting as a reseller. A **tenant** is a client that uses a fraud prevention application provided by a landlord. Tenants have the possibility to create, manage and deploy their own rules and lists depending on the permissions the landlord has given to them.

Duplicated for Tenant

When a rules project is set as **Duplicated for Tenant** it means that the set of rules applies to all the tenants in the same way. In terms of deployment, a rule snapshot is created and applied in runtime, the events are then grouped by each tenant and all rules defined in the rule snapshot are applied for each tenant individually, as illustrated in the following image.

By Tenant

When a rules project is set as **By Tenant**, it means that a set of rules applies to specific tenants. In terms of deployment, a rule snapshot is created and applied in runtime, the events are then grouped by tenant and the corresponding set of rules are applied for each tenant, as illustrated in the following image.

Advanced vs. simple rules

Users can set the type to advanced or simple, the former being the default type.

This setting has the following meaning:

- **Advanced rules projects** contain rules that can be written using a template and which may have an environment and state.
- **Simple rules projects** contain rules that are not editable inside Pulse and may be used for self-service rules created in Case Manager.

When a rules project is set as *Simple*, the *State* input is hidden, the label for the *Actions* becomes *Secondary Actions*, and the multi-tenancy options are limited to *Global* and *By Tenant*.

Self-service rules

With **self-service rules** (SSR), fraud analysts can write and deploy rules themselves very quickly.

The characteristics of self-service rules in a nutshell are as follows:

- They are deployed directly to the Production environment, in real time.
- They are created in Case Manager by analysts and/or their managers.
- Governance can audit and support self-service rules.

This lets you implement completely new processes and achieve higher operational efficiencies while dynamically following changes in fraud patterns.

Testing self-service rules

Self-service rules can be tested in Case Manager, allowing production environments to execute Spark jobs processing the most recent events that have gone through a self-service rule block of a given workflow. These events will be tested against an updated version of that rule block and measure *Hit Rate*, *Money Captured*, etc.

In order for Pulse to process the data in the same way as when going through the live workflow, those events are stored in a custom data store used only for self-service rule testing.

This data store thus receives every event that goes through a self-service rule block in the workflow. Depending on the testing configuration in Case Manager, this data store keeps the data of the time interval that is necessary for rule testing.

Auditing self-service rules

Pulse can be configured to keep audit logs of the creation, edition, deletion, ordering, testing, and publishing operations made to rules. The raw logs are accessible via a REST API through a query that specifies a rule, a workflow element or an application. Get in touch with your contact person at Feedzai to get more information on the audit logs and the Self-Service Rules REST API.